

02 — Threads, Blocks, Grids e Mapeamento no Hardware

Pré-requisito: [01 — modelo de execução](#).

1. A hierarquia lógica

```
kernel launch
├── grid
│   ├── block 0
│   │   └── threads
│   ├── block 1
│   │   └── threads
│   └── ...
```

Uma **thread** executa o programa do kernel para um índice. Um **block** agrupa threads que podem cooperar por shared memory e barreiras. Um **grid** contém todos os blocks de um launch.

Índice global 1D clássico:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

Essas variáveis descrevem coordenadas lógicas; não dizem em qual SM específico o trabalho vai executar.

2. O mapeamento físico

- Um block é atribuído inteiro a **um SM** e permanece ali até terminar.
- Um SM pode manter vários blocks residentes se houver threads, registers, shared memory e slots de block suficientes.
- Blocks diferentes podem executar em qualquer ordem. Um kernel comum não deve depender de ordem entre blocks.
- Dentro do SM, threads de um block são divididas em warps de 32.

```
grid com muitos blocks
    ↓ distribuidor
SM0: [block 3][block 17]    SM1: [block 0]    SM2: [block 8][block 9] ...
```

O índice do block não indica ordem temporal nem SM.

3. Arredondamento em warps

O número de warps alocados é:

```
warps/block = ceil(threads/block ÷ 32)
```

Um block com 100 threads usa 4 warps. O último tem 4 lanes ativas e 28 lanes sem trabalho. O hardware ainda reserva/escalona o warp como unidade.

Pergunta rápida

Um kernel lança 56 blocks de 256 threads na RTX 3060. Isso garante exatamente 2 blocks por SM durante toda a execução?

► Resposta

Confusões comuns

- Thread CUDA não é thread de sistema operacional.
- “Block grande” não significa automaticamente melhor uso da GPU.
- `cudaDeviceSynchronize()` no host e `__syncthreads()` no device têm escopos diferentes.
- Um block não pode usar shared memory de outro block num kernel comum.

O que guardar

- Grid/block/thread é a decomposição lógica; SM/warp/lane é a execução física.
- Block é unidade de colocação; warp é unidade de emissão de instrução.
- Sempre arredonde threads para cima ao contar warps.

Conexão: agora que threads viraram warps, precisamos entender como uma instrução é emitida e como o scheduler escolhe trabalho pronto.